

ARTIFICIAL INTELLIGENCE – PHASE 3

CREATE A CHATBOT IN PYTHON

PROBLEM STATEMENT:

The challenge is to create a Chatbot in Python that provides a exceptional customer service, answering user queries on a website or an application.

- Deliver high quality support to the users
- Ensuring a positive user experience
- Customer satisfaction

REQUIRED LIBRARIES:

Install the required libraries:

- TensorFlow
- Keras
- NLTK (Natural Language Toolkit)

You can install these libraries using the following command:

```
pip install tensorflow keras nltk
```

1.Loading and Preprocessing Data

To get started, let's load our training data from a JSON file named 'intents.json'. This file contains predefined intents, patterns, and responses that our chatbot will use to learn.

```
import nltk
from nltk.stem import WordNetLemmatizer
import json
import pickle
# Load intents from JSON file
data_file = open('intents.json').read()
```

```

intents = json.loads(data_file)
# Preprocess data
words = []
classes = []
documents = []
ignore_words = ['?', '!']
# Tokenize and preprocess patterns
for intent in intents['intents']:
    for pattern in intent['patterns']:
        w = nltk.word_tokenize(pattern)
        words.extend(w)
        documents.append((w, intent['tag']))
        if intent['tag'] not in classes:
            classes.append(intent['tag'])
# Lemmatize words and preprocess classes
# ...

```

2.Creating Training Data

Let's prepare the training data by creating a bag-of-words representation for each pattern. The bag-of-words approach involves creating a binary array indicating the presence or absence of words from the vocabulary in the pattern. We'll also create an output array corresponding to the intent for each pattern.

```

# Create training data
training = []
output_empty = [0] * len(classes)
for doc in documents:
    bag = []
    pattern_words = doc[0]
    pattern_words = [lemmatizer.lemmatize(word.lower()) for word in
pattern_words]

```

```

# Create bag of words
for w in words:
    bag.append(1) if w in pattern_words else bag.append(0)

# Create output array
output_row = list(output_empty)
output_row[classes.index(doc[1])] = 1
training.append([bag, output_row])
# ...

```

3. Building and Compiling the Model

We will now build our neural network model using Keras. The model consists of input, hidden, and output layers. The input layer size is determined by the vocabulary size. We use the softmax activation function in the output layer for multiclass classification. The model is compiled with the SGD optimizer and categorical cross-entropy loss function.

```

from keras.models import Sequential
from keras.layers import Dense, Dropout
from keras.optimizers import SGD

# Build the model
model = Sequential()
model.add(Dense(128, input_shape=(len(train_x[0]),), activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(64, activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(len(train_y[0]), activation='softmax'))

# Create SGD optimizer
sgd = SGD(learning_rate=0.01, momentum=0.9, nesterov=True)

# Compile the model
model.compile(loss='categorical_crossentropy', optimizer=sgd,
metrics=['accuracy'])
# ...

```

4.Training the Model

It's time to train our model using the prepared training data. We'll train for a specified number of epochs with a defined batch size. After training, we'll save the model to a file named 'chatbot_model.h5'.

```
# Train the model and save it
hist = model.fit(np.array(train_x), np.array(train_y), epochs=200,
batch_size=5, verbose=1)
model.save('chatbot_model.h5', hist)
# ...
```

5.Loading the Model and Defining Response Functions

Now, let's load the trained model and define functions for preprocessing user input and generating responses. We'll predict the intent of the input using the trained model and choose a response from predefined intents.

```
# Load the trained model
model = load_model('chatbot_model.h5')
# Define response functions
# ...
def chatbot_response(text):
    ints = predict_class(text, model)
    res = getResponse(ints, intents)
    return res
# ...
```